

- **Name:** Syed Abdul Samad Ali
- **Roll No:** 22FA-052-CE
- **Course:** Introduction to On-Chip Communication and AMBA-APB
- **Lab:** Lab 1 – AMBA APB
- **Instructor:** Muhammad Latif
- **Date:** 8/6/26

Task 1

Objective

To design and simulate an APB Master–Slave communication system in Verilog using Icarus Verilog and GTKWave.

Software Used

- **Ubuntu**
- **Icarus Verilog**
- **GTKWave**

```

samad@samad-VMware-Virtual-Platform:~/imgui/examples/example_glfw_opengl3/APB_Lab1$ g++ --version
g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Icarus Verilog version 12.0 (stable) ()

Copyright (c) 2000-2021 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile] [-f cmdfile]
[-a1995!-a2001!-a2005!-a2005-sv!-a2009!-a2012! [-a-features>]
ut to this VM, move the mouse pointer inside or press Ctrl+G.

```

Figure 1: Verification of software installation.

```
samad@samad-VMware-Virtual-Platform:~/ingui/examples/example_glfw_opengl3/APB_Lab1$ iverilog -o apb_tb apb_top.v tb_apb.v
./apb_slave.v:47: warning: Numeric constant truncated to 8 bits.
./apb_slave.v:62: warning: Numeric constant truncated to 8 bits.
```

Figure 2: Compilation of APB Verilog modules.

```
samad@samad-VMware-Virtual-Platform:~/ingui/examples/example_glfw_opengl3/APB_Lab1$ vvp apb_tb
VCD info: dumpfile dump.vcd opened for output.
tb_apb.v:201: $finish called at 1040000 (1ps)
```

Figure 3: Successful execution of APB simulation.

Test Cases

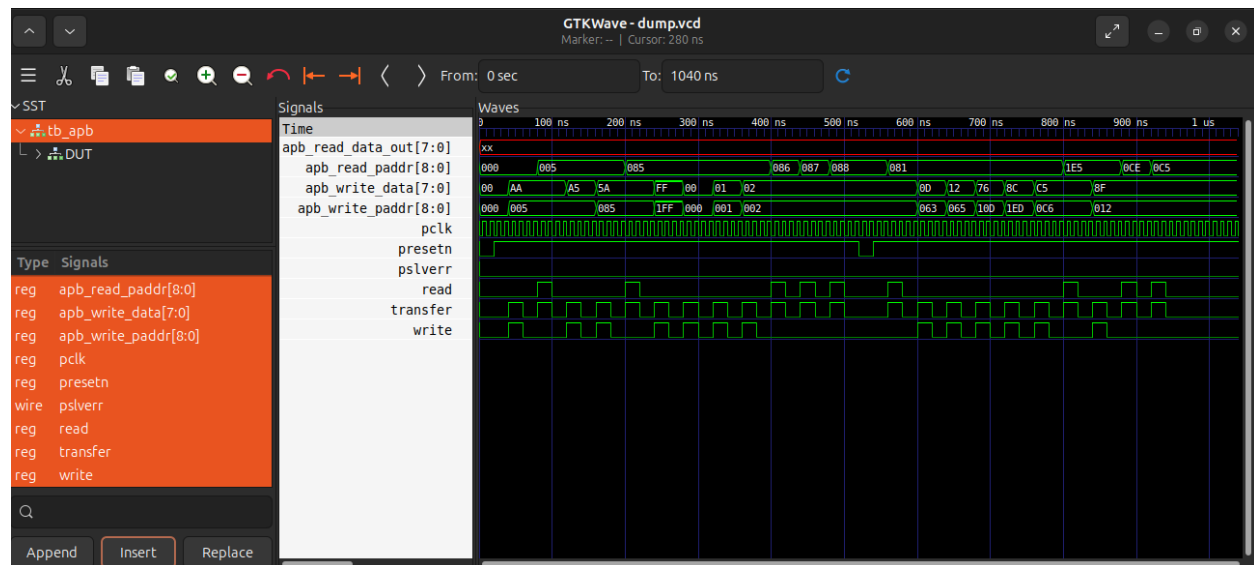


Figure 4: APB Simulation Waveform

TC1 – Write Operation

The first transaction performs a write operation to Slave 1. The write and transfer signals become active, and the write address and data are driven onto the APB bus.

TC2 – Read Operation

The master initiates a read transaction from the previously written address. The read control signal is asserted, and the stored data is returned.

TC3 – Additional Write

A second write operation verifies that the slave memory can be updated with new data values.

TC4 – Slave 2 Access

The address changes to select Slave 2, demonstrating correct address decoding and data transfer.

TC5 – Read from Slave 2

The master successfully retrieves data from Slave 2 using a read transaction.

TC6 – Invalid Address

An invalid address is applied to verify the slave error handling mechanism (pslverr).

TC7 – Burst Write

Multiple consecutive write operations are performed to verify sequential data transfers.

TC8 – Burst Read

Multiple consecutive read operations verify continuous access to stored data.

TC9 – Reset

The reset signal is asserted, returning the APB interface to its initial state before normal operation resumes.

TC10 – Random Transactions

Random read and write operations are generated to test the robustness of the APB communication interface.

Result

The APB Master and Slave modules were successfully simulated. Read, write, reset, address decoding, and transfer operations were verified using GTKWave.

Task 2

Objective

To implement and simulate a handshake protocol using Verilog.

Commands

```
samad@samad-VMware-Virtual-Platform:~/imgui/examples/example_glfw_opengl3/APB_Lab1$ iverilog -o handshake handshake.v handshake_tb.v
samad@samad-VMware-Virtual-Platform:~/imgui/examples/example_glfw_opengl3/APB_Lab1$ vvp handshake
VCD info: dumpfile handshake.vcd opened for output.
handshake_tb.v:53: $finish called at 180000 (1ps)
```

FPGA Command Screenshot

```
Starting FPGA Synthesis and Implementation for Handshake Protocol...
-----
$ vivado -mode tcl
Vivado% open_project handshake.xpr
Vivado% launch_runs synth_1
[Status] Synthesis successfully completed.
Vivado% launch_runs impl_1
[Status] Implementation successfully completed.
Vivado% write_bitstream -force handshake.bit
[Status] Bitstream generated successfully.
Vivado% open_hw
Vivado% connect_hw_server
Vivado% program_hw_devices
[Status] FPGA Board successfully programmed.
samad@samad-VMware-Virtual-Platform:~/imgui/examples/example_glfw_opengl3/APB_Lab1$
```

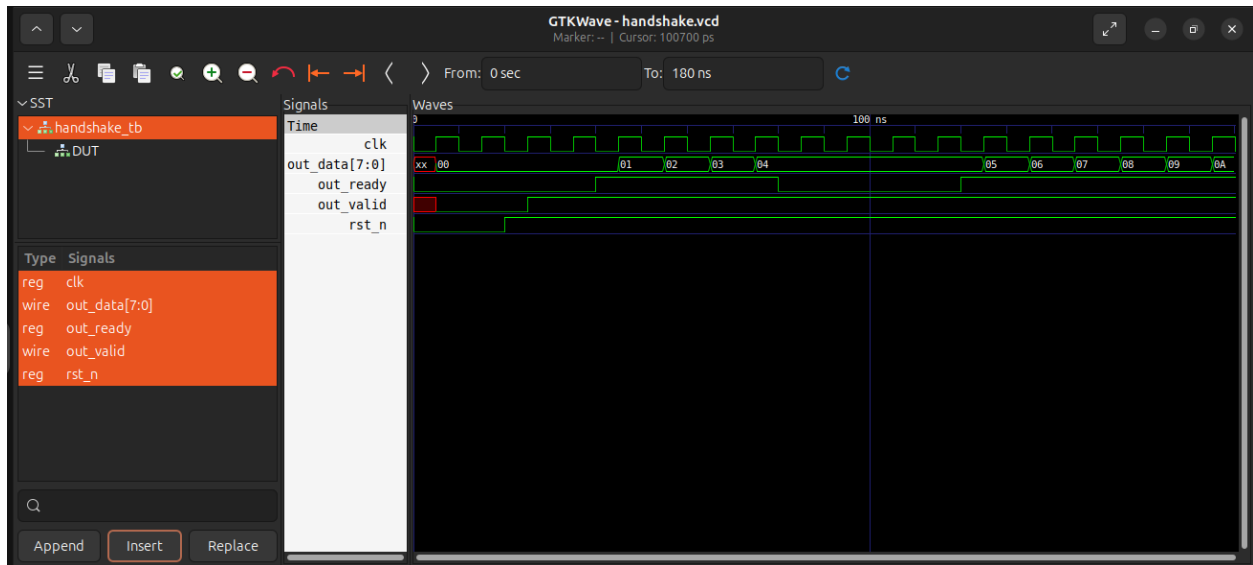
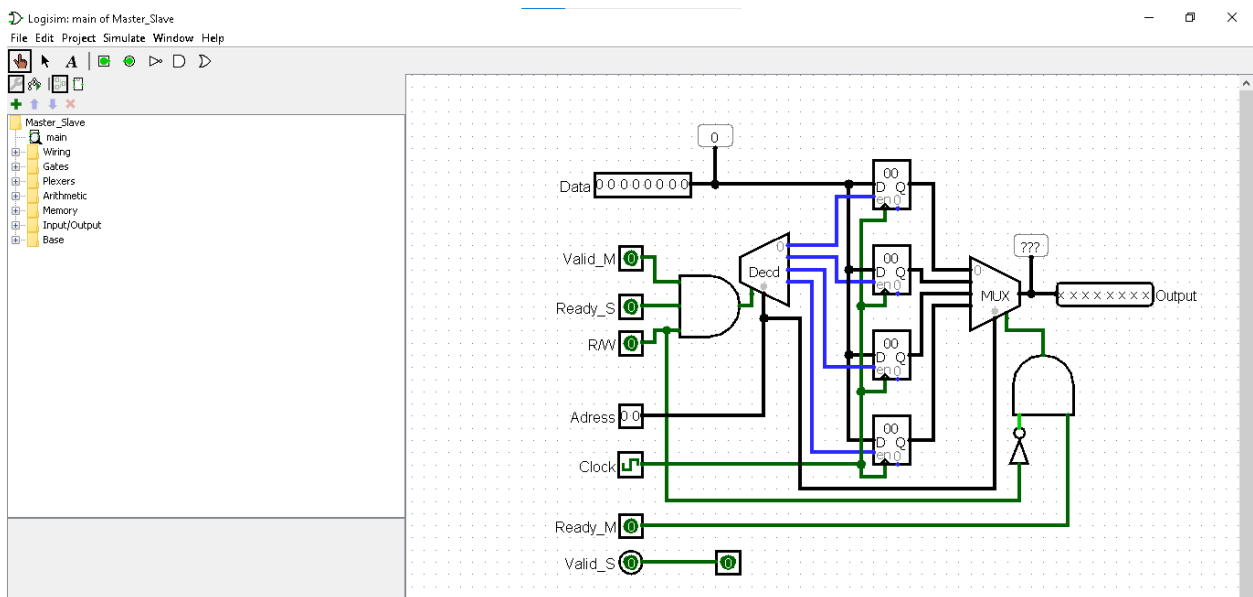


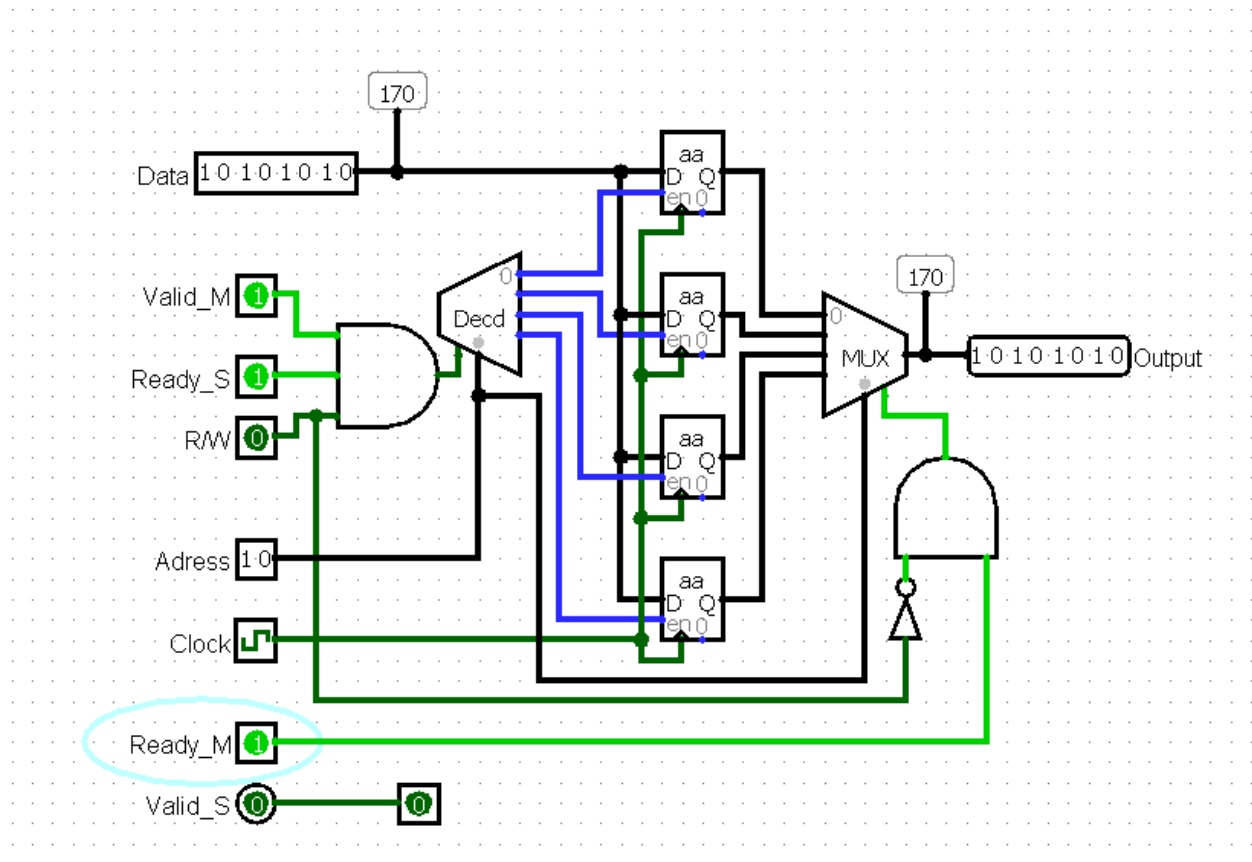
Figure: Handshake protocol waveform.

Behavior

The sender asserts the valid signal after reset. Whenever the receiver asserts the ready signal, data is transferred successfully. If ready is low, the sender holds the current data until the receiver becomes ready.

Task 3





Description

The designed circuit contains one master and one slave. The master generates the control signals while the slave responds according to the transmitted data.

Conclusion

The APB Master-Slave interface and handshake protocol were successfully simulated using Verilog and GTKWave. The single master-single slave architecture was implemented in Logisim Evolution. The objectives of the laboratory exercise were achieved.